

DevOps Lab (BCSL657D)

Viva Questions & Answers

All 12 Experiments — Complete Reference

Brindavan College of Engineering | VI Semester| Dept of CSE

Prepared By: Prof. Sruthi Krishna U

Experiment 1 — Maven & Gradle: Overview, Key Differences, Installation

Q1. What is a build automation tool?

Ans: A build automation tool automates repetitive tasks in software development like compiling source code, managing dependencies, running tests, packaging applications, and deploying software. Examples include Maven, Gradle, and Ant.

Q2. What is Apache Maven?

Ans: Apache Maven is a project management and build automation tool that uses an XML-based configuration file called pom.xml. It follows 'convention over configuration' and uses Maven Central Repository for dependency management.

Q3. What is Gradle?

Ans: Gradle is a flexible, modern build automation tool that uses Groovy or Kotlin DSL for configuration. It supports incremental builds and caching, making it faster than Maven. It is widely used for Java and Android projects.

Q4. What are the key differences between Maven and Gradle?

Ans: Maven uses XML (pom.xml) while Gradle uses Groovy/Kotlin DSL (build.gradle). Gradle is faster due to incremental builds and caching. Maven is easier to learn (convention over config), while Gradle is more flexible. Maven follows a fixed lifecycle; Gradle allows custom tasks.

Q5. What is JAVA_HOME and why is it set?

Ans: JAVA_HOME is an environment variable that points to the JDK installation directory. Build tools like Maven and Gradle use this variable to locate the Java runtime. Without it, commands like mvn and gradle may fail.

Q6. What command verifies Maven installation?

Ans: The command 'mvn -version' verifies Maven installation. It displays Maven version, Java version, operating system, and the Java home directory if installation is successful.

Q7. What is the Gradle Wrapper?

Ans: The Gradle Wrapper (gradlew / gradlew.bat) is a script that downloads and uses a specific Gradle version. It ensures all developers and CI systems use the same Gradle version without requiring a prior installation of Gradle.

Experiment 2 — Working with Maven: POM File, Dependencies, Plugins

Q1. What is pom.xml?

Ans: pom.xml stands for Project Object Model. It is Maven's main configuration file containing project information (groupId, artifactId, version), dependencies, build plugins, and lifecycle configuration. It is located at the root of every Maven project.

Q2. What are groupId, artifactId, and version in Maven?

Ans: groupId identifies the organization (e.g., com.vtu), artifactId is the project name (e.g., devops-lab), and version specifies the release number (e.g., 1.0-SNAPSHOT). Together they uniquely identify a Maven artifact in any repository.

Q3. What is Maven Central Repository?

Ans: Maven Central Repository is a public online storage location where Java libraries (JAR files) are hosted. When you declare a dependency in pom.xml, Maven automatically downloads it from this repository and stores a local copy in ~/.m2/repository.

Q4. What does 'mvn clean install' do?

Ans: 'mvn clean' deletes the target/ folder (previous build output). 'mvn install' runs compile, test, and package phases and then installs the artifact into the local Maven repository (~/.m2). Running them together ensures a fresh, complete build.

Q5. What is the Maven Compiler Plugin?

Ans: The Maven Compiler Plugin compiles Java source code. By default it is automatically bound to the compile phase. It can be configured in pom.xml to specify the Java version (e.g., maven.compiler.source=11).

Q6. What is the standard Maven directory structure?

Ans: src/main/java/ holds source code, src/main/resources/ holds configuration files, src/test/java/ holds test code, and target/ is the build output folder created automatically by Maven.

Q7. What is the Maven Surefire Plugin used for?

Ans: The Maven Surefire Plugin runs unit tests during the 'test' phase of the Maven build lifecycle. It automatically discovers and executes test classes following naming conventions (ending in Test, Tests, or TestCase).

Experiment 3 — Working with Gradle: Build Scripts, DSL, Task Automation

Q1. What is a Gradle task?

Ans: A task is the basic unit of work in Gradle. It represents a single action like compiling code, running tests, or copying files. Tasks can depend on other tasks, forming a Directed Acyclic Graph (DAG) that Gradle executes in the correct order.

Q2. What is Groovy DSL vs Kotlin DSL in Gradle?

Ans: Groovy DSL uses `build.gradle` (dynamic scripting, concise). Kotlin DSL uses `build.gradle.kts` (statically typed, better IDE auto-completion and type safety). Kotlin DSL is preferred for large projects due to better tooling support.

Q3. What does 'gradle build' do?

Ans: 'gradle build' compiles source code, runs tests, and packages the application. It executes multiple tasks in sequence: classes (compile), test (run tests), and jar/assemble (package). The output is placed in the `build/libs/` folder.

Q4. What is the application plugin in Gradle?

Ans: The application plugin in Gradle adds the ability to run a Java main class using 'gradle run'. It requires specifying the `mainClass` property. It also creates distribution archives with start scripts.

Q5. What is a DAG in Gradle context?

Ans: DAG stands for Directed Acyclic Graph. Gradle builds a task execution graph where each task may depend on other tasks. Tasks execute in the correct dependency order (no cycles) ensuring build correctness and enabling parallel execution.

Q6. How do you add an external dependency in Gradle?

Ans: In `build.gradle`, inside the `dependencies` block, add: `implementation 'groupId:artifactId:version'`. For example: `implementation 'com.google.guava:guava:33.0.0-jre'`. Gradle downloads it from the repositories declared (e.g., `mavenCentral()`).

Q7. What is the difference between 'gradle clean' and 'gradle build'?

Ans: 'gradle clean' deletes the `build/` directory (removes old outputs). 'gradle build' compiles code, runs tests, and packages the application. Running 'gradle clean build' together ensures a clean, fresh rebuild from scratch.

Experiment 4 — Practical: Build with Maven & Migrate to Gradle

Q1. What is Maven archetype:generate?

Ans: 'mvn archetype:generate' creates a new Maven project from a template (archetype). The 'maven-archetype-quickstart' template creates a standard Java project with pom.xml, App.java, and AppTest.java. It is the fastest way to start a new Maven project.

Q2. Why might gradle build fail after migrating from Maven?

Ans: Gradle may fail if a test source set exists (src/test/) but has no valid tests or conflicting dependencies. Solution: remove src/test/ (rmdir /s src\test on Windows) and rebuild. Also, the Java compiler version mismatch between Maven and Gradle settings can cause failures.

Q3. What does mainClass = 'com.example.App' do in Gradle?

Ans: It configures the entry point for the 'gradle run' command. The application plugin uses this property to know which class contains the main() method to execute when 'gradle run' is invoked.

Q4. What is the purpose of 'gradle init' during Maven to Gradle migration?

Ans: 'gradle init' initializes a Gradle build in the current directory. When run inside an existing Maven project, it detects the pom.xml and offers to generate a Gradle build from it. It creates build.gradle, settings.gradle, and the Gradle wrapper files.

Q5. What is the difference between target/ (Maven) and build/ (Gradle)?

Ans: Both are output directories. Maven uses target/ for compiled classes and JAR files. Gradle uses build/ (with subdirectories build/classes/ and build/libs/). Both are automatically cleaned by their respective clean commands.

Experiment 5 — Jenkins: Installation & First-Time Configuration

Q1. What is Jenkins?

Ans: Jenkins is an open-source automation server used primarily for Continuous Integration (CI) and Continuous Delivery (CD). It automates building, testing, and deploying software by monitoring source code repositories and triggering pipelines on code changes.

Q2. What is Continuous Integration (CI)?

Ans: Continuous Integration is a DevOps practice where developers frequently commit code to a shared repository (multiple times a day). Each commit automatically triggers a build and test process, helping detect errors quickly and reducing integration problems.

Q3. What is initialAdminPassword in Jenkins?

Ans: initialAdminPassword is a randomly generated password stored at C:\ProgramData\Jenkins\jenkins\secrets\initialAdminPassword (Windows). It is required to unlock Jenkins for the first time after installation through the web browser.

Q4. What are Jenkins plugins?

Ans: Plugins extend Jenkins functionality. They integrate Jenkins with version control systems (Git plugin), build tools (Maven Integration plugin), notification services, cloud platforms, and testing frameworks. Jenkins has 1800+ available plugins.

Q5. What is the default port for Jenkins?

Ans: Jenkins defaults to port 8080. During installation, a custom port (e.g., 8085) can be chosen if 8080 is already in use. The Jenkins dashboard is then accessed at [http://localhost:\[port\]](http://localhost:[port]).

Q6. What is a Jenkins Freestyle project?

Ans: A Freestyle project is the simplest type of Jenkins job. It allows configuring build steps sequentially without using code (pipeline scripts). It supports source code management, build triggers, build steps (shell, Maven, etc.), and post-build actions through the GUI.

Q7. What is 'Install suggested plugins' in Jenkins setup?

Ans: It installs the most commonly used Jenkins plugins automatically during initial setup. These include plugins for Git, Pipeline, Credentials, Email notification, and Maven/Gradle integration. It is recommended for beginners to get a functional Jenkins environment quickly.

Experiment 6 — Continuous Integration with Jenkins

Q1. What is a Jenkins Pipeline?

Ans: A Jenkins Pipeline is a suite of plugins that supports implementing and integrating CI/CD pipelines. It uses a Groovy-based DSL (Jenkinsfile) to define build stages. Pipelines can be Declarative (structured, recommended) or Scripted (flexible, advanced).

Q2. What is a Declarative Pipeline in Jenkins?

Ans: A Declarative Pipeline uses the 'pipeline {}' block syntax and defines stages with stage() and steps with steps {}. It is simpler, more readable, and preferred for most use cases. It validates syntax before running and has built-in support for post conditions.

Q3. What does 'agent any' mean in a Jenkinsfile?

Ans: 'agent any' instructs Jenkins to run the pipeline on any available agent (executor). Jenkins will assign the first free executor from its pool. Alternatives include 'agent none' (define per stage) or 'agent { label 'linux' }' (specific node).

Q4. What is 'mvn clean install' used for in Jenkins pipeline?

Ans: 'mvn clean install' is the most common Maven command used in CI pipelines. It deletes previous build output (clean), compiles code, runs all tests, packages the JAR, and installs it to the local Maven repository. A failure in any phase stops the pipeline.

Q5. How does Jenkins integrate with GitHub?

Ans: Jenkins connects to GitHub through the Source Code Management section of a job configuration. Enter the repository URL and credentials. Jenkins can poll for changes (SCM polling) or receive webhooks from GitHub to trigger builds automatically on every commit.

Q6. What is the significance of Console Output in Jenkins?

Ans: Console Output shows the real-time log of a Jenkins build execution. It displays all commands run, Maven/Gradle output, test results, errors, and the final BUILD SUCCESS or BUILD FAILURE message. It is the primary tool for debugging failed builds.

Q7. What does a red ball in Jenkins indicate?

Ans: A red ball (or red icon) indicates that the last build failed — either the code did not compile, tests failed, or a pipeline stage returned a non-zero exit code. Blue/green indicates success, and yellow indicates an unstable build (tests failed but compilation succeeded).

Experiment 7 — Configuration Management with Ansible

Q1. What is Ansible?

Ans: Ansible is an open-source automation tool developed by Red Hat used for configuration management, application deployment, and task automation. It is agentless — it communicates with target hosts via SSH without installing any software on them. Configuration is written in YAML.

Q2. What does 'agentless' mean in Ansible?

Ans: Agentless means Ansible does not require any software (agent) to be installed on the target machines being managed. It uses SSH (for Linux) or WinRM (for Windows) to connect and execute tasks remotely, making it lightweight and easy to adopt.

Q3. What is an Ansible Inventory?

Ans: An inventory is a file (inventory.ini or inventory.yml) that lists the hosts Ansible will manage. Hosts can be grouped (e.g., [webservers], [dbservers]) and can have variables assigned. The inventory is passed to commands with the -i flag.

Q4. What is an Ansible Playbook?

Ans: A playbook is a YAML file that describes automation tasks. It contains one or more plays, each targeting a group of hosts and running an ordered list of tasks. Each task uses a module to perform an action (create file, install package, start service, etc.).

Q5. What is an Ansible Module?

Ans: An Ansible module is a pre-built unit of work that performs a specific task on a host. Common modules include: ping (test connectivity), copy (copy files), apt/yum (manage packages), service (control services), file (manage directories/permissions), and command/shell (run commands).

Q6. What does 'become: yes' mean in Ansible?

Ans: 'become: yes' enables privilege escalation — Ansible runs tasks as a superuser (root) using sudo. This is required when tasks need elevated permissions, such as installing packages, creating system directories, or modifying system services.

Q7. What is ansible_connection=local in inventory?

Ans: ansible_connection=local tells Ansible to run tasks locally (on the control node itself) without SSH. This is used in lab environments like Killercoda where the control node and managed node are the same machine.

Q8. What is PLAY RECAP in Ansible output?

Ans: PLAY RECAP is the summary printed at the end of a playbook run. It shows for each host: ok (tasks that ran without changes), changed (tasks that modified the system), unreachable (connection failed), and failed (task returned error). All values should be 0 for failed in a successful run.

Experiment 8 — Jenkins CI Pipeline + Ansible Deployment

Q1. What is CI/CD?

Ans: CI (Continuous Integration) automatically builds and tests code on every commit. CD (Continuous Delivery/Deployment) automatically deploys verified builds to target environments. Together, CI/CD creates a fast, reliable software delivery pipeline from code commit to production.

Q2. What is 'mvn clean package'?

Ans: 'mvn clean package' first deletes the target/ directory (clean phase), then compiles the source code, runs tests, and creates a deployable JAR or WAR file in target/. It is commonly used in CI pipelines to produce a fresh build artifact.

Q3. What is an artifact in CI/CD context?

Ans: An artifact is the deployable output of a build process. In Maven Java projects, it is typically a JAR or WAR file in the target/ directory (e.g., hello-app-1.0-SNAPSHOT.jar). Artifacts are stored in artifact repositories and deployed by tools like Ansible, Jenkins, or Azure Pipelines.

Q4. What does the Ansible 'copy' module with remote_src: yes do?

Ans: When remote_src: yes is set, the copy module copies a file from one location to another on the same remote machine (instead of from the control node to the remote). In this experiment, it copies the JAR from /root/hello-app/target/ to /opt/myapp/ on localhost.

Q5. What is the purpose of 'register' in an Ansible task?

Ans: 'register' captures the output of a task into a variable. In the deployment playbook, 'register: result' saves the output of the 'ls -lh' command. The next task uses 'debug: msg: {{ result.stdout }}' to display the captured output.

Experiment 9 — Azure DevOps: Overview & Account Setup

Q1. What is Azure DevOps?

Ans: Azure DevOps is a Microsoft cloud-based platform providing end-to-end tools for software development: Azure Repos (version control), Azure Pipelines (CI/CD), Azure Boards (work tracking), Azure Test Plans (testing), and Azure Artifacts (package management).

Q2. What is Azure Repos?

Ans: Azure Repos provides Git-based version control hosted in Microsoft Azure. Teams can create repositories, branch, create pull requests, and review code. It supports both Git (distributed) and TFVC (Team Foundation Version Control, centralized) workflows.

Q3. What is Azure Boards?

Ans: Azure Boards is a work item tracking tool supporting Agile, Scrum, and Kanban methodologies. Teams can create user stories, bugs, tasks, and epics, assign them to sprints, and track progress on Kanban boards and burndown charts.

Q4. What is the difference between Azure Pipelines and Jenkins?

Ans: Azure Pipelines is a cloud-hosted CI/CD service with Microsoft-managed build agents, tight Azure integration, and YAML-based configuration stored in Git. Jenkins is a self-hosted open-source server requiring manual setup, plugin management, and infrastructure maintenance. Azure Pipelines is easier for cloud-native projects.

Q5. What is the purpose of initializing a repository with a README?

Ans: Initializing with a README creates the first commit, establishing the main branch. Without this, the repository is empty and cannot be used with pipelines (which need at least one branch/commit). The README also serves as project documentation.

Experiment 10 — Azure Build Pipelines, Unit Tests, Reports

Q1. What is a YAML pipeline in Azure DevOps?

Ans: A YAML pipeline is a pipeline defined as code in a .yaml file stored in the repository (usually azure-pipelines.yaml). It defines triggers, the agent pool, and steps. Since it lives in the repo, it is versioned, reviewable, and reproducible — following the 'pipeline as code' principle.

Q2. What is 'trigger: - main' in YAML?

Ans: 'trigger: - main' configures the pipeline to run automatically every time code is pushed to the main branch. This is the core CI trigger — any commit to main starts the pipeline, runs tests, and reports success or failure.

Q3. What is vmImage: ubuntu-latest?

Ans: vmImage: ubuntu-latest specifies that the pipeline runs on a Microsoft-hosted Ubuntu Linux agent. Microsoft provides and maintains this virtual machine. It comes pre-installed with common tools like Java, Gradle, Maven, Node.js, and Docker.

Q4. What is the PublishTestResults@2 task?

Ans: PublishTestResults@2 is an Azure Pipelines built-in task that uploads test result XML files to Azure DevOps. After the pipeline runs, results appear under the Tests tab showing pass/fail counts, test names, duration, and failure details. It supports JUnit, NUnit, and xUnit formats.

Q5. What does 'condition: succeededOrFailed()' do?

Ans: It makes the task run regardless of whether previous steps succeeded or failed. This is important for publishing test results — you want to see test failures even when the overall build fails. Without this condition, the publish step would be skipped on failure.

Q6. Why use 'gradle build' instead of './gradlew build' in Azure Pipelines?

Ans: On Microsoft-hosted ubuntu-latest agents, Gradle is pre-installed system-wide, so 'gradle build' works directly. './gradlew' requires the Gradle Wrapper files (gradlew, gradle/wrapper/) to be committed to the repository, which adds extra files. For simple lab setups, 'gradle build' is sufficient.

Experiment 11 — Release Pipelines, Azure App Service, Continuous Deployment

Q1. What is Azure App Service?

Ans: Azure App Service is a fully managed Platform-as-a-Service (PaaS) for hosting web applications, REST APIs, and mobile backends. It supports multiple languages (Node.js, Python, Java, PHP, .NET). It handles OS updates, scaling, load balancing, and security patches automatically.

Q2. What is Continuous Deployment (CD)?

Ans: Continuous Deployment is a practice where every code change that passes all automated tests is automatically deployed to production without human intervention. In Azure Pipelines, 'trigger: - main' in YAML implements this — every push to main triggers a full build and deploy cycle.

Q3. What is the ArchiveFiles@2 task?

Ans: ArchiveFiles@2 compresses files into a ZIP archive. In the release pipeline, it packages the entire repository into app.zip stored in \$(Build.ArtifactStagingDirectory). Azure App Service requires a ZIP file for deployment via the AzureWebApp@1 task.

Q4. What is AzureWebApp@1 task?

Ans: AzureWebApp@1 is an Azure Pipelines built-in task that deploys a web application to Azure App Service. It requires azureSubscription (service connection name), appName (App Service name), and package (ZIP file path). It handles authentication and deployment automatically.

Q5. What is a Service Connection in Azure DevOps?

Ans: A Service Connection is a secure, trusted link between Azure DevOps and an external service (e.g., Azure subscription, GitHub, Docker Hub). It stores credentials safely so pipelines can deploy resources without hardcoding secrets in YAML files.

Q6. What is Azure Key Vault?

Ans: Azure Key Vault is a cloud service for securely storing and managing secrets (passwords, API keys, connection strings), encryption keys, and certificates. Pipelines can retrieve secrets at runtime using the AzureKeyVault@2 task without exposing them in code or YAML.

Q7. How is rollback performed in Azure App Service?

Ans: Rollback can be done by redeploying a previous build artifact. In Azure Pipelines, trigger an older pipeline run and redeploy. Azure App Service also supports deployment slots (staging/production swap) and can be configured to keep previous deployment packages for quick rollback.

Experiment 12 — Complete DevOps Pipeline: Best Practices & Wrap-Up

Q1. What are the 6 stages of a complete DevOps pipeline?

Ans: The 6 stages are: (1) Code & Commit — developers push code to version control; (2) Build — code is compiled/packaged; (3) Test — automated tests run; (4) Code Quality Scan — static analysis checks security and quality; (5) Deploy to Staging — human review before production; (6) Deploy to Production — live deployment after approval.

Q2. What is GitHub Actions?

Ans: GitHub Actions is a CI/CD platform built into GitHub. Workflows are defined in YAML files (.github/workflows/*.yml). They trigger on events (push, PR, schedule), run on GitHub-hosted virtual machines, and execute steps (checkout, build, test, deploy). It is free for public repositories.

Q3. What is Docker and why is it used in DevOps?

Ans: Docker is a containerization platform that packages applications and their dependencies into containers — lightweight, portable units that run consistently across any environment. In DevOps, Docker ensures 'it works on my machine' becomes 'it works everywhere' by eliminating environment differences between development, testing, and production.

Q4. What is a Dockerfile?

Ans: A Dockerfile is a text file with instructions to build a Docker image. Each instruction (FROM, RUN, COPY, CMD, etc.) adds a layer to the image. 'docker build -t name .' builds the image from the Dockerfile in the current directory. 'docker run name' starts a container from that image.

Q5. What is the most important DevOps best practice?

Ans: The most important practice is 'never skip tests to meet a deadline.' A bug shipped to production costs 10x more to fix than one caught in testing. Other key practices: commit small and often, keep pipeline configuration in version control, automate repetitive tasks, monitor after deployment, and maintain strict environment separation.

Q6. What is a rollback in DevOps?

Ans: A rollback is reverting a deployed application to a previous stable version when a new deployment causes issues. It is a critical safety net in production. Good CI/CD pipelines keep previous deployment artifacts and can trigger a rollback automatically (via health checks) or manually (by redeploying a specific commit).

Q7. What is the difference between Continuous Delivery and Continuous Deployment?

Ans: Continuous Delivery means every build that passes tests is ready to be deployed to production — but requires a manual approval step before actual deployment. Continuous Deployment goes one step further: every passing build is automatically deployed to production without human intervention.

Q8. What is SonarQube / SonarCloud used for in DevOps?

Ans: SonarQube (self-hosted) and SonarCloud (cloud-hosted) are static code analysis tools used in the 'Code Quality Scan' stage of a pipeline. They analyze code for bugs, security vulnerabilities, code smells, and coverage gaps, providing a quality gate that can fail the pipeline if thresholds are not met.